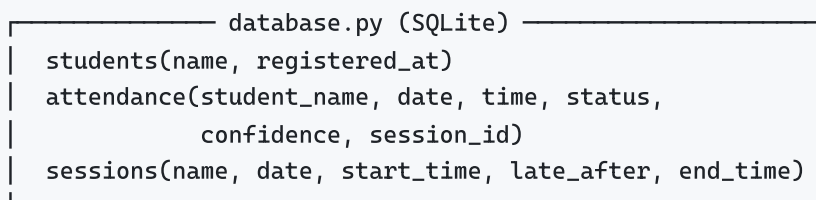
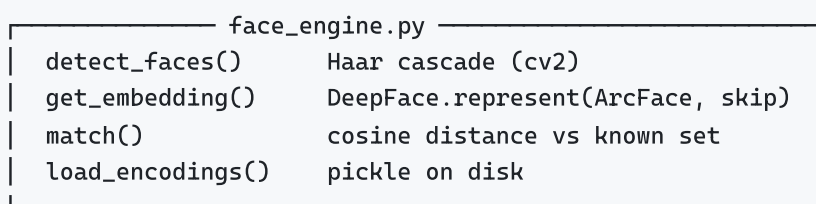
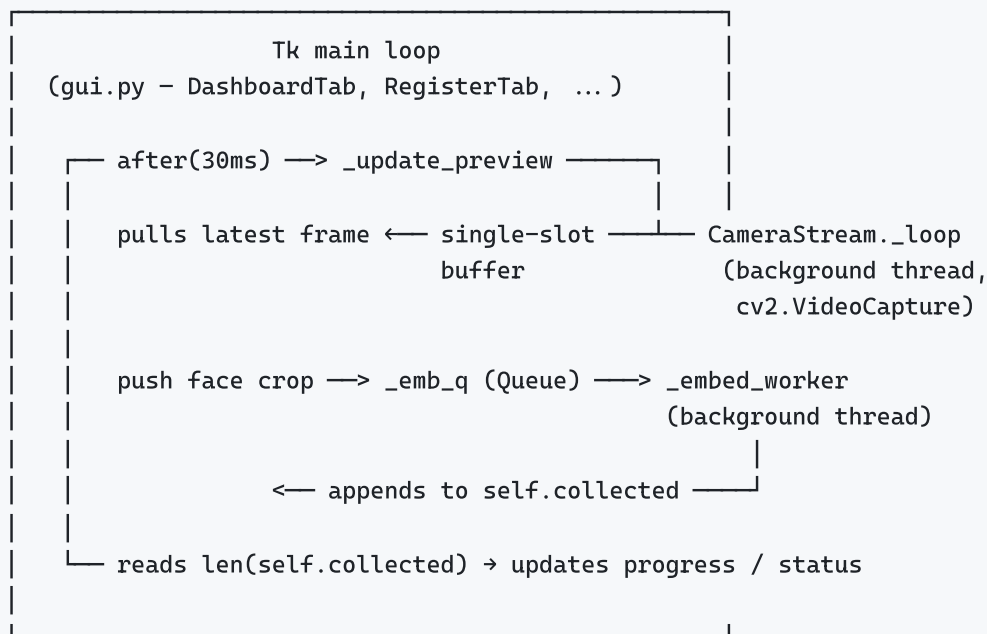


FaceGuard — Developer Documentation

This document covers the architecture, data model, and module reference of FaceGuard. For installation and end-user instructions see [README.md](#).

1. High-level architecture



Key design points:

- **The Tk main thread never blocks on inference.** ArcFace embedding takes ~270 ms on CPU; doing that synchronously inside the `after(30, ...)` callback would freeze the GUI. Frames flow: *capture thread* → *buffer* → *main thread (detect + draw)* → *embedding worker*.

- **Single-slot frame buffer.** The capture thread overwrites a single field protected by a lock. The main thread always gets the freshest frame and never accumulates lag.
 - **Bounded embedding queue.** `_emb_q = Queue(maxsize=2)` and we use `put_nowait()` — if the worker is still processing the previous crop we drop the new one. ArcFace is the bottleneck, not face detection.
 - **DLL bootstrap before TF imports.** TensorFlow's import-time DLL check fails on some Windows installs that lack the standalone VC++ redistributable. `face_engine.py` calls `os.add_dll_directory()` on `venv\Scripts` *before* `from deepface import DeepFace`, so we can ship the runtime alongside the app rather than relying on a system install.
-

2. Module reference

`main.py`

Entry point. Parses `--cli` and dispatches to either `gui.main()` or the legacy text menu. The text menu is preserved so the project still runs on machines without a display (over SSH, etc.).

`config.py`

Plain constants — no functions. See the table in [README.md → Configuration](#).

`config` is also imported by `gui.py` so the Settings tab can mutate values **at runtime** (for the current launch). Persist by editing this file.

`database.py`

SQLite wrapper. Schema:

```

CREATE TABLE students (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  name        TEXT UNIQUE NOT NULL,
  registered_at TEXT DEFAULT (datetime('now','localtime'))
);

CREATE TABLE attendance (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  student_name TEXT NOT NULL,
  date        TEXT NOT NULL,
  time        TEXT NOT NULL,
  status      TEXT NOT NULL DEFAULT 'present', -- present | late | absent
  confidence  REAL,
  session_id  INTEGER,
  UNIQUE(student_name, date)
);

CREATE TABLE sessions (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  name        TEXT NOT NULL,
  date        TEXT NOT NULL,
  start_time  TEXT NOT NULL,
  late_after  TEXT,
  end_time    TEXT
);

```

`init_db()` does a `PRAGMA table_info(attendance)` check and runs additive `ALTER TABLE` statements for `status` / `confidence` / `session_id` when upgrading from the original two-column schema, so existing databases keep working.

Public functions:

Function	Purpose
<code>init_db()</code>	Create schema, migrate columns if needed
<code>add_student(name)</code> / <code>delete_student(name)</code>	Insert / remove a student row
<code>get_all_students()</code> / <code>get_student_names()</code>	Read all students
<code>mark_attendance(name, status, confidence, session_id)</code>	Idempotent per <code>(student, date)</code> — returns <code>False</code> if duplicate
<code>get_attendance(date=None)</code>	All rows or rows for a specific <code>YYYY-MM-DD</code>
<code>get_distinct_dates()</code>	For the date filter in Reports
<code>get_per_student_summary()</code>	<code>(name, present, late, absent, total)</code> across all sessions
<code>start_session(name, late_after)</code> / <code>end_session(id)</code>	Open a session row / close it and auto-mark absentees
<code>get_active_session()</code> / <code>get_recent_sessions(limit)</code>	For Dashboard / future history view

The connection is created per call (`_conn`) — SQLite handles concurrency fine for this workload and it avoids cross-thread `sqlite3.Connection` gotchas.

`face_engine.py`

The compute layer.

Function	Notes
<code>detect_faces</code>	Haar cascade. Optional <code>scale</code> to downsample first then rescale coords.
<code>get_embedding</code>	<code>DeepFace.represent(... detector_backend='skip')</code> — face is pre-cropped.
<code>match</code>	Cosine distance against the known set, returns <code>(name, distance)</code> .
<code>load_encodings</code> / <code>save_encodings</code>	Pickled <code>{"names": [...], "encodings": [...]}</code> .
<code>warm_up</code>	Forces ArcFace model load + the first slow inference.
<code>_ensure_msvc_runtime</code>	Adds <code>venv\Scripts</code> to the DLL search path before TF is imported.

The pickle file lives at `data/encodings/known_faces.pkl` . Encodings are 512-d ArcFace vectors, L2-normalised so cosine distance $\equiv \frac{1}{2}$ -euclidean².

`register.py`

CLI enrolment loop. Captures up to `MIN_SAMPLES` single-face frames, averages embeddings, L2-normalises, and writes the result. Also handles `--remove`.

`recognize.py`

CLI recognition loop. Detection runs every 3rd frame; `match()` decides name + status. `mark_attendance()` is idempotent per `(student, date)` so a face appearing many times in one session only writes one row.

`report.py`

CLI report menu. Pretty-prints tables and writes CSV. Superseded by the Reports tab in the GUI but kept for headless / scripted use.

`gui.py`

The Tkinter app. Structure:

- `CameraStream` — owns a `cv2.VideoCapture`, a background thread that reads frames into a single-slot buffer, and a `start()/stop()` lifecycle that can be repeated (the original `WebcamThread` subclassed `threading.Thread`, which can only be started once — that's been fixed).
- `_open_camera()` — tries `CAP_DSHOW`, then `CAP_MSMF`, then the default backend. Some webcams only enumerate under a subset of these.
- `cv2_to_tk(frame)` — `BGR → RGB → PIL.Image → ImageTk.PhotoImage`.
- One class per tab (`DashboardTab`, `RegisterTab`, ...). Each owns its own `CameraStream` so `register` and `recognise` can run independently.
- `RegisterTab._embed_worker` — daemon thread that pulls face crops from `self._emb_q` and appends embeddings to `self.collected`. The main thread reads `len(self.collected)` to drive the progress bar and `Capturing ... X / N` status text.

The Tk main thread:

- Polls `cam.get_frame()` every 30 ms via `self.after(30, _update_preview)`.
- Runs Haar detection (fast, <50 ms).
- Pushes crops to the worker queue (bounded, drops on full).
- Reads collected count, updates UI, redraws the preview image.

3. Data model — life of a row

Enrolment

1. User clicks Capture in RegisterTab.
2. `_update_preview` runs every 30 ms. When it sees exactly one face, the crop is put on `_emb_q`.
3. `_embed_worker` does `get_embedding(crop)` and appends to `self.collected` (only while `self.capturing` is true and we're below `MIN_SAMPLES`).
4. Once `len(self.collected) == MIN_SAMPLES`, `_finish_capture()` runs on the Tk thread: - Mean of all 512-d vectors → L2 normalise. - Append to the encodings pickle. - `add_student(name)` inserts the student row.

Live recognition

1. User clicks **Start session** in AttendanceTab.
2. `start_session(name, late_after)` inserts a row into `sessions` and returns its id.
3. For every recognised face whose distance $\leq$ `COSINE_THRESHOLD`: - Status decided by `status_for_time(now, late_after)`. - `mark_attendance(name, status, confidence, session_id)` — inserts a row, or returns `False` if there's already a row for `(name, date)`.
4. On **End session**: - `end_session(id)` sets `end_time` on the session row. - Finds every student with no `attendance` row for that date. - Inserts an `absent` row for each.

4. Performance notes

Measured on a CPU-only Windows laptop (no AVX-512, no GPU):

Operation	Time
Haar face detection (640×480)	~10–30 ms
<code>DeepFace.represent</code> (ArcFace)	~270 ms / call
Direct <code>model.__call__</code>	~280 ms / call
10-sample enrolment (worker)	~3–5 s end-to-end
Live recognition tick (every 3rd)	~80 ms (amortised)

ArcFace inference is the dominant cost. Speed-ups available:

- Use `tensorflow[and-cuda]` and a CUDA GPU → typically <30 ms.
 - Switch `MODEL_NAME` to `Facenet` (128-d, faster but slightly less accurate).
 - Drop `MIN_SAMPLES` to 6 — embeddings are averaged, so quality plateaus quickly past ~8 samples for a stationary subject.
-

5. Configuration knobs in one place

File / class	What you can tune
<code>config.py</code>	Thresholds, sample count, late cutoff (persistent).
GUI Settings tab	The same values — in memory for the current launch.
<code>_CAPTURE_BACKENDS</code>	Order in which webcam backends are tried.
<code>_emb_q = Queue(maxsize=2)</code> (RegisterTab)	Backpressure: bigger = more queueing, smaller = drops more frames.
<code>frame_idx % 3</code> (AttendanceTab)	Recognition cadence vs preview cadence trade-off.

6. Troubleshooting deep dive

Black preview, no error

The webcam opened but produced no frames. `CameraStream` retries three backends and surfaces the failure list via `last_error`. If even `default` fails, another process is holding the webcam — close Skype / Teams / browser tabs and try again.

TensorFlow can't import

`face_engine._ensure_msvc_runtime()` calls `os.add_dll_directory(<venv>\Scripts)` if `msvcp140.dll` lives there. If that fails too you'll see TF's own error message pointing at https://aka.ms/vs/17/release/vc_redist.x64.exe. Install that, restart, done.

Capture progress never moves

Likely cause: ArcFace weights never downloaded — silent failures eat the crops in the worker. Check `%USERPROFILE%\deepface\weights\arcface_weights.h5` exists and is ~130 MB. If not, download manually from: https://github.com/serengil/deepface_models/releases/download/v1.0/arcface_weights.h5.

Everyone is "Unknown"

`COSINE_THRESHOLD` is too tight or registration was poor. Raise it to 0.45, or re-enrol with varied head poses and lighting.

Multiple people in frame at registration

Registration intentionally requires *exactly one* face — having two people visible halts sample collection until one steps out.

7. Roadmap

Near-term

- [] **Class roster import** — read a CSV of expected names and pre-populate students without webcam capture (useful for bulk onboarding).
- [] **Re-register / append samples** instead of remove-and-add.
- [] **Confidence histogram** in Reports to help tune `COSINE_THRESHOLD` empirically.
- [] **Dark theme** for the GUI (Sun Valley ttk theme already supports this).
- [] **Audible / on-screen feedback** when a face is recognised in a live session — currently it's silent.
- [] **Per-session reports** filtered by `session_id`, not just by date.
- [] **Settings persistence** — write the Settings tab back to `config.py` instead of mutating in memory.

Medium-term

- [] **Multi-camera support** — entry door and exit door, separate `CameraStream` instances pushing into the same session.
- [] **Anti-spoofing** — blink detection or a small liveness model so a printed photo doesn't get marked present. DeepFace already has `anti_spoofing=True` in newer releases — wire it up.
- [] **GPU path** — `pip install tensorflow[and-cuda]` and document requirements; should drop embedding to <30 ms and make `MIN_SAMPLES=20+` effortless.
- [] **Switchable detector** — wire `config.DETECTOR_BACKEND` through to actually pick between Haar / RetinaFace / MediaPipe. Right now the constant exists but isn't consumed.
- [] **Timetable / schedule** — instead of clicking Start session, sessions open automatically at 09:00, 11:00, etc. from a `timetable.json`.

Long-term

- [] **Web dashboard** — FastAPI + Vite/Svelte front-end so a teacher can pull up the attendance grid from any browser on the LAN.
- [] **Notifications** — email/SMS to a parent or admin when a student is marked absent.
- [] **Centralised multi-classroom mode** — one server, multiple capture kiosks, single source of truth in Postgres.
- [] **Mobile registration** — phone uploads a few photos via a QR-coded URL, no need to bring everyone past the kiosk.

- [] **Privacy options** — purge embeddings on a schedule, encrypt the pickle at rest, opt-in consent flow.
 - [] **Packaging** — `pyinstaller` build so non-developers get a single `FaceGuard.exe` instead of needing Python and a venv.
-

8. Contributing / extending

The codebase is small (~1 kLOC) and intentionally flat — no plugin system, no DI container. To extend:

- **New report view** → add a method to `report.py` and a tab to `gui.py`.
- **New face model** → set `MODEL_NAME` in `config.py`. DeepFace will fetch weights on first call. Match logic stays the same — only the embedding dimensionality changes (handled transparently by cosine distance).
- **New status** (e.g. *excused*) → add it to the `status` column's allowed values, extend `_status_for_time` and the summary query.
- **Persist Settings tab** → write the values back to `config.py` with `ast` / `configparser`. The Settings tab already collects them in `apply()`.